The `std::thread` class is used to manage a thread of execution. It provides a means of starting a new thread of execution and waiting for the completion of a thread of execution. It also provides a means for identifying and provides other functions for managing threads of execution.

Class definition

```
class thread
{
public:
    // Types
    class id;
    typedef implementation-defined native_handle_type; // optional

    // Construction and Destruction
    thread() noexcept;
    ~thread();

    template<typename Callable,typename Args...>
    explicit thread(Callable&& func,Args&&... args);

    // Copying and Moving
    thread(thread const& other) = delete;
    thread(thread&& other) noexcept;

    thread& operator=(thread const& other) = delete;
    thread& operator=(thread&& other) noexcept;

    void swap(thread& other) noexcept;

    void join();
    void detach();
    bool joinable() const noexcept;

    id get_id() const noexcept;

    native_handle_type native_handle();

    static unsigned hardware_concurrency() noexcept;
};

void swap(thread& lhs,thread& rhs);
```

Std::Thread::Id Class

An instance of `std::thread::id` identifies a particular thread of execution.